Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology

School of Computing

Department of Computer Science and Engineering

10211CS224 / FULL STACK APPLICATION DEVELOPMENT (JAVA)

PROJECT REVIEW : MILESTONE PROJECT – 1

Date: 25-03-2026

Real-Time Event Synchronization Engine

SDG 9 – Industry, Innovation and Infrastructure

Presented By:

SRIDHAR R – VTU24391 – 23UECS0544

Courses Handling Faculty:

Dr. M.MAGESHWARI Designation

INTRODUCTION

1
2
3
4
5
6
7
8
9

- involve multiple users and devices interacting with shared data simultaneously, leading to synchronization challenge Modern applications es.
- Traditional systems rely on manual refresh or polling, causing delays, high latency, and data inconsistency.
- This project is important because it ensures instant data updates across all connected clients without delay.
- It improves system efficiency, reduces network load, and enhances overall user experience.
- Real-time synchronization is widely used in chat apps, multiplayer games, stock trading platforms, and IoT systems.
- For example, platforms like Google Docs allow multiple users to edit documents simultaneously in real time.
- The system works using an event-driven architecture where events are detected, processed, and broadcast instantly.
- Overall, the Real-Time Event Synchronization Engine ensures fast, reliable, and consistent communication in modern distributed systems.

PROBLEM STATEMENT

1
2
3
4
5
6
7
8
9

Problem Statement:

- Modern applications involve multiple users and devices accessing and modifying shared data simultaneously, leading to synchronization conflicts.
- Traditional polling-based systems cause delays, increased network traffic, and inefficient resource utilization.
- Data inconsistency occurs when updates made by one user are not instantly reflected to others.
- High latency in distributed systems negatively affects real-time applications like gaming, live chat, and IoT monitoring.
- Concurrent data modifications may result in data loss, duplication, or overwriting issues.
- Systems without real-time synchronization provide poor user experience due to manual refresh requirements.
- Scalability becomes difficult when handling thousands of real-time events simultaneously.
- Therefore, there is a need for a scalable, efficient, and low-latency Real-Time Event Synchronization Engine that ensures consistent and instant data updates across distributed systems.

OBJECTIVE(S)

1
2
3
4
5
6
7
8
9

- To design and develop a scalable Real-Time Event Synchronization Engine that enables instant data updates across multiple connected clients with low latency.

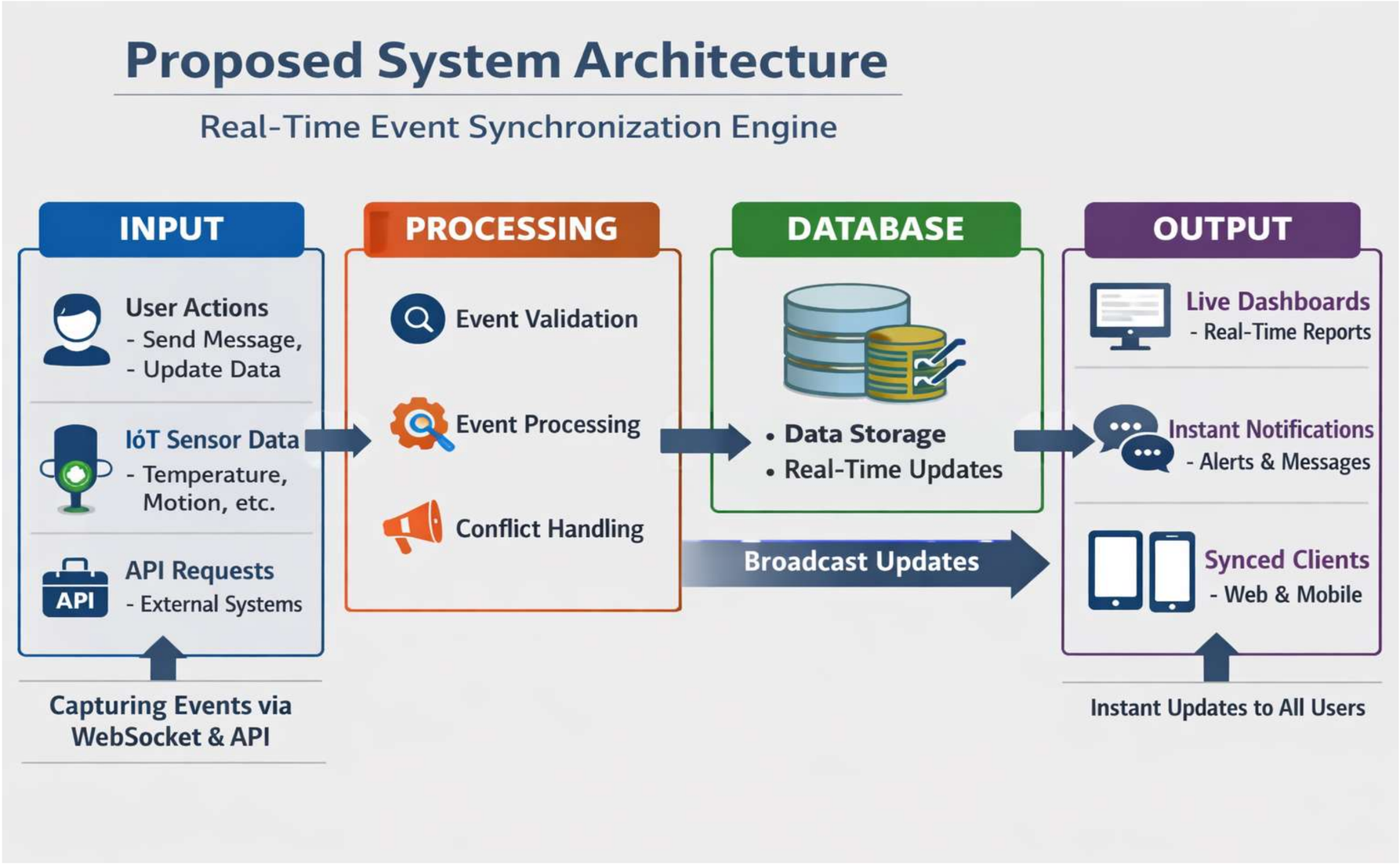
- To evaluate the performance of the synchronization system based on response time, data consistency, and scalability under concurrent user events.

PROPOSED SYSTEM

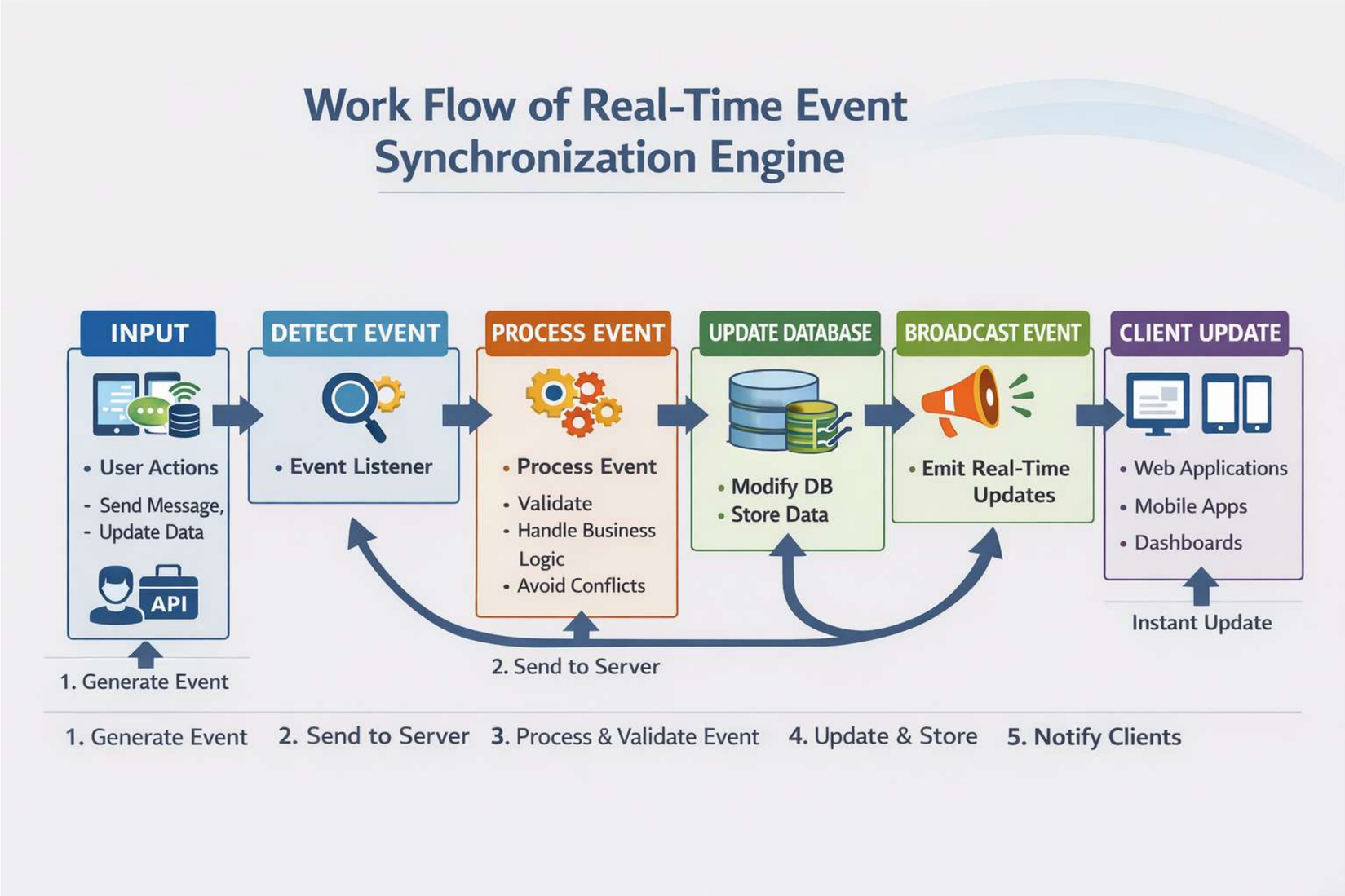
1
2
3
4
5
6
7
8
9

- The proposed system is an event-driven Real-Time Event Synchronization Engine that captures, processes, and broadcasts events instantly across connected clients.
- It uses technologies like WebSockets or message queues to enable real-time, bidirectional communication between server and clients.
- The system detects user or device events, validates them, updates the database if required, and synchronizes changes immediately.
- It solves data inconsistency issues by ensuring all users view the same updated information at the same time.
- It reduces latency and eliminates manual refresh by replacing traditional polling mechanisms.
- The architecture supports high scalability, allowing multiple concurrent users or devices to operate efficiently.
- It improves network efficiency by transmitting data only when an event occurs, reducing unnecessary traffic.
- Compared to existing systems, it offers faster response time, better reliability, improved user experience, and easier maintenance.

PROPOSED SYSTEM ARCHITECTURE



WORKFLOW


Work Flow of Real-Time Event Synchronization Engine
INPUT
• User Actions
- Send Message,
- Update Data
API
1. Generate Event
DETECT EVENT
• Event Listener
PROCESS EVENT
• Process Event
• Validate
• Handle Business Logic
• Avoid Conflicts
2. Send to Server
UPDATE DATABASE
• Modify DB
• Store Data
BROADCAST EVENT
• Emit Real-Time Updates
CLIENT UPDATE
• Web Applications
• Mobile Apps
• Dashboards
Instant Update
1. Generate Event
2. Send to Server
3. Process & Validate Event
4. Update & Store
5. Notify Clients

TECHNOLOGIES USED

1
2
3
4
5
6
7
8
9

Frontend:
- HTML5
- CSS3,
- JavaScript

Backend
- Node.js
- Server.js

Database:
- My SQL

Tools:
- Notepad
- Web Browser (Chrome)
- Postman (optional for testing)

Hardware Requirements:
- 4GB RAM minimum (8GB recommended)
- Basic laptop/desktop
- Internet connection

Software Requirement:
- Windows / Linux
- Node.js installed
- MongoDB installed

MODULE DESCRIPTION

Module 1: Input Module

This module captures events generated by users or devices. Examples: sending a message, updating data, or IoT sensor readings. It forwards the event to the server through WebSocket or API.

Module 2: Event Processing Module

This module validates and processes the received event. It applies business logic and checks for errors or conflicts. After processing, the event is prepared for storage and broadcasting.

Module 3: Database Module

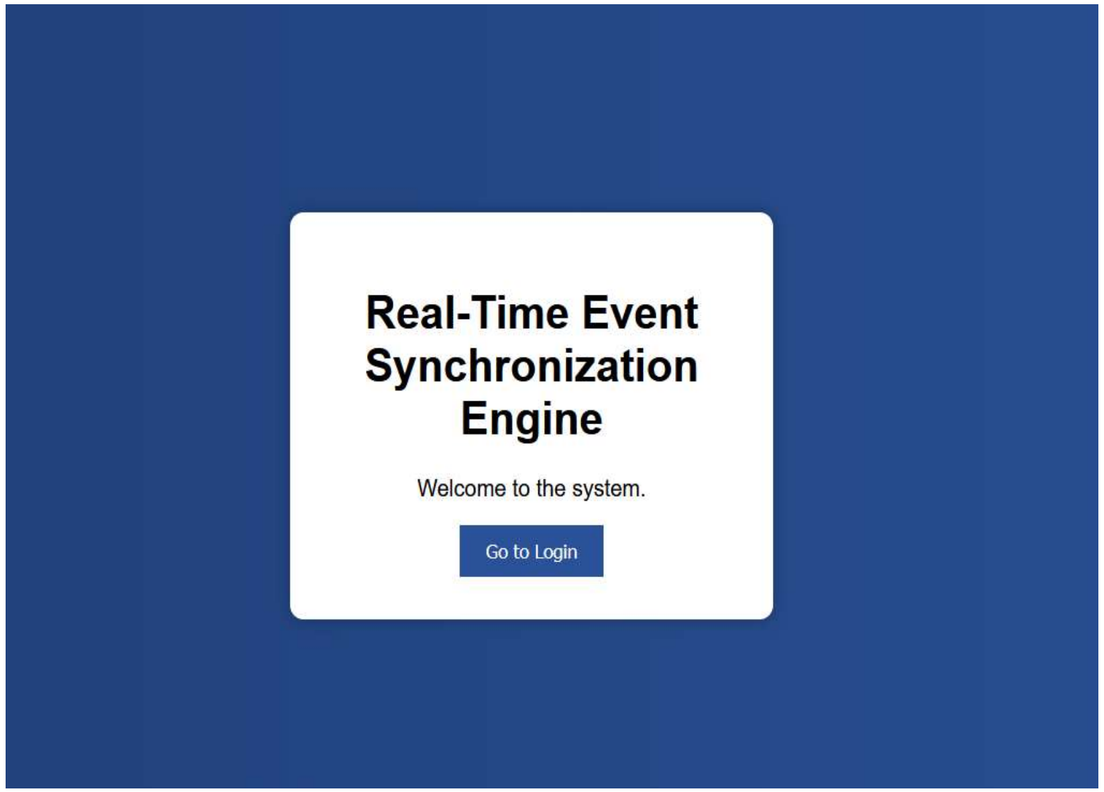
This module stores and updates the synchronized data. It ensures data consistency and maintains records of events. The database supports real-time read and write operations.

Module 4: Output / Broadcast Module

This module sends updated data to all connected clients instantly. User interfaces refresh automatically without manual reload. It ensures all users see the same updated information at the same time.

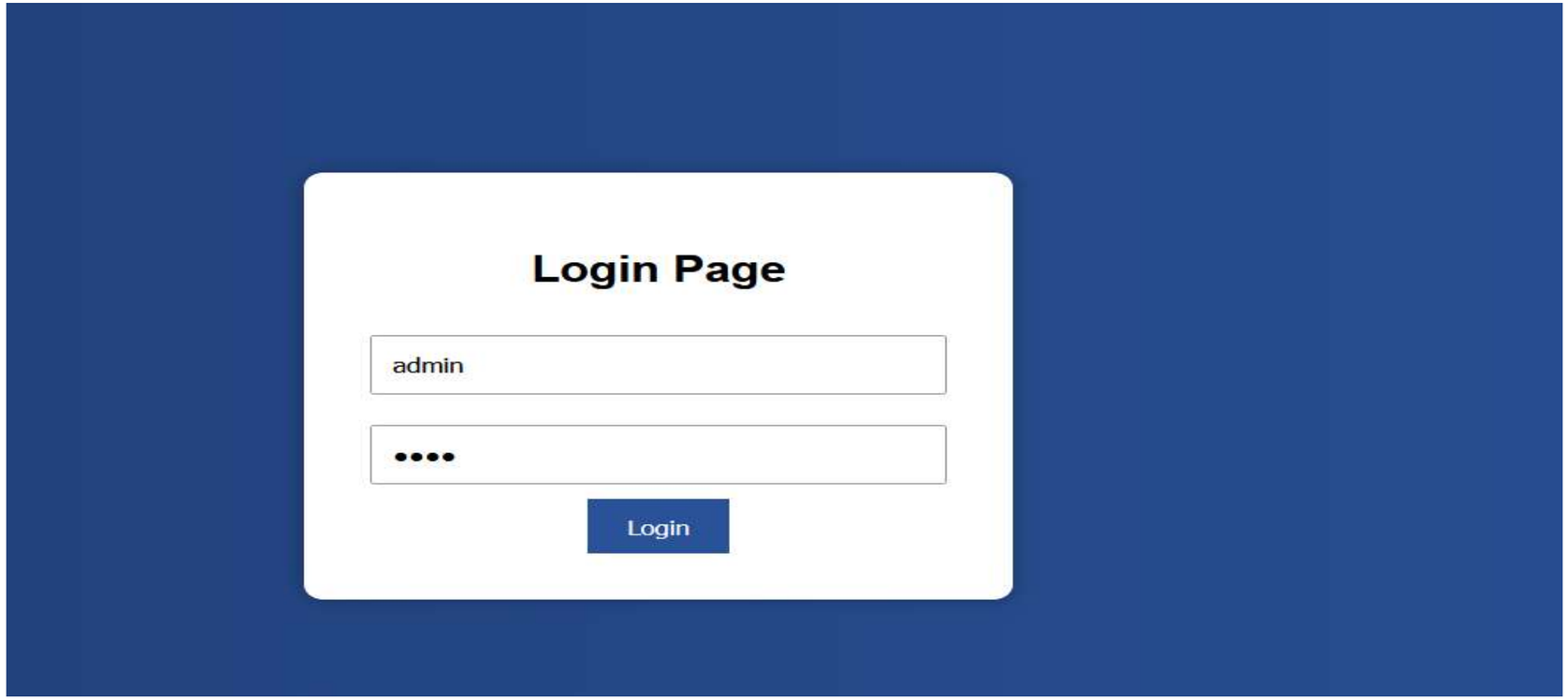
IMPLEMENTATION SCREENSHOTS

Home Page : The Home Page serves as the entry point of the Real-Time Event Synchronization Engine, providing users with an introduction to the system and its purpose. It offers a simple and user-friendly interface that guides users to proceed to the login page for secure access. This page ensures smooth navigation and creates a professional first impression of the application. By clicking the "Go to Login" button, users can move to the authentication page to access the event synchronization features.



IMPLEMENTATION SCREENSHOTS

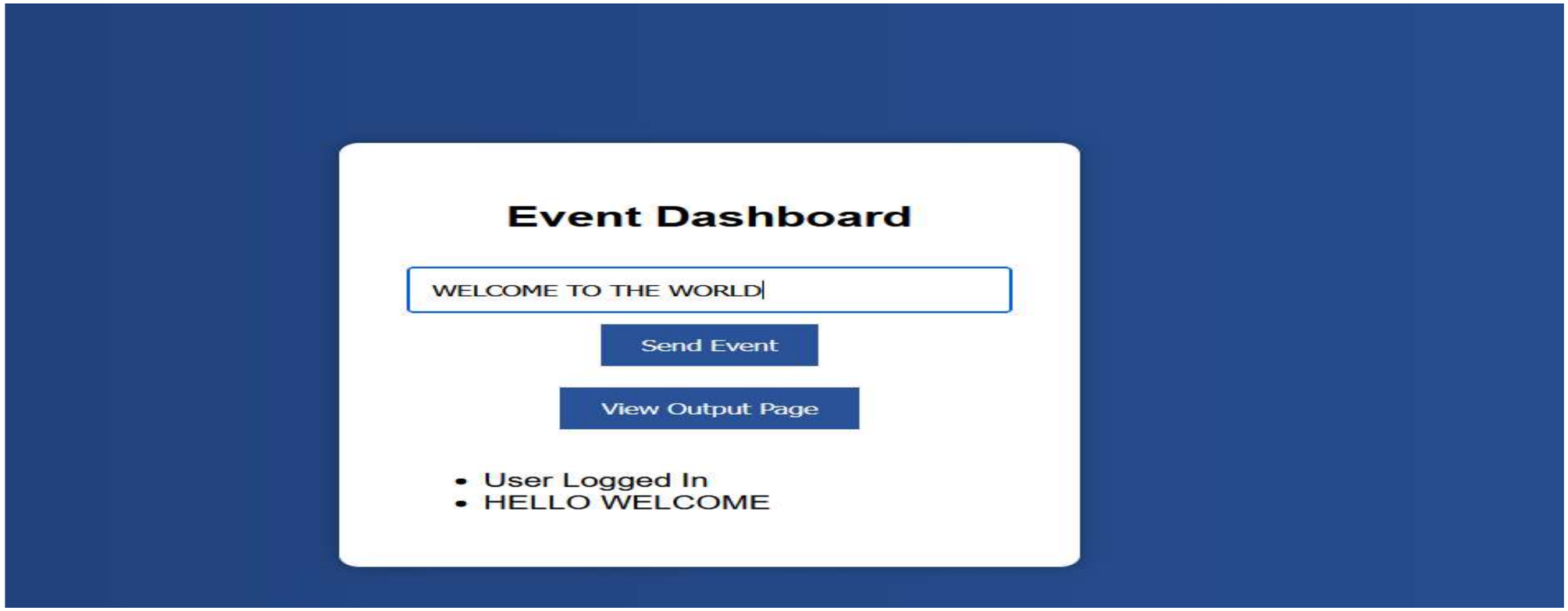
Login page : The Login Page is used to authenticate users before granting access to the system. It requires a valid username and password to ensure that only authorized users can enter the Real-Time Event Synchronization Engine. Once the correct credentials are entered, the user is redirected to the dashboard where they can create and view synchronized events securely.



IMPLEMENTATION SCREENSHOTS

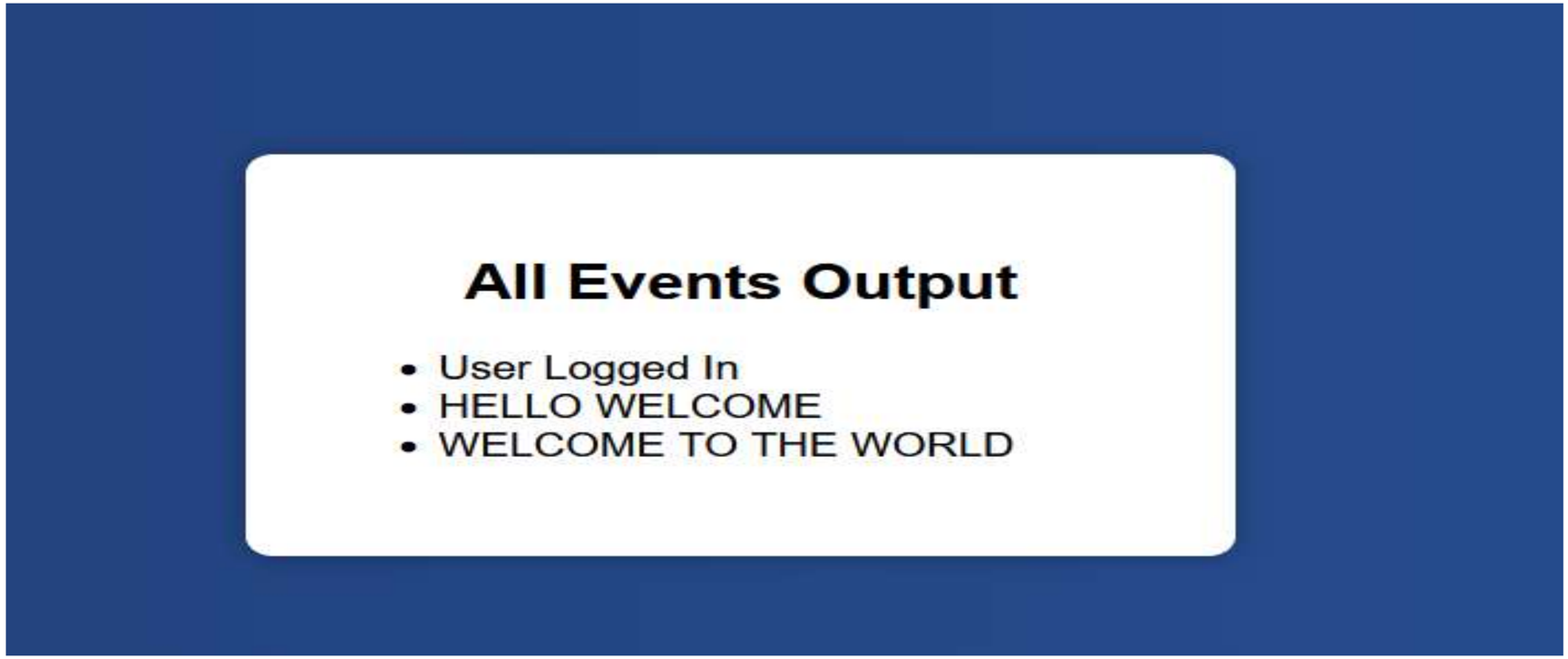
1
2
3
4
5
6
7
8(B)
9

Main Module: The Main Module (Event Dashboard) is the core part of the Real-Time Event Synchronization Engine where users can create and manage events. It allows users to enter event messages, send them to the system, and view synchronized updates in real-time. This module connects the frontend and backend, storing events temporarily and displaying them dynamically without refreshing the page. It also provides navigation to the Output Page to view all recorded events clearly.



IMPLEMENTATION SCREENSHOTS

The **Output Page** displays all the events that have been added in the system through the dashboard. It shows the synchronized event messages in a clear list format, allowing users to view all recorded activities in one place. This page automatically updates at regular intervals, ensuring that newly added events are displayed without manually refreshing the browser, thereby demonstrating real-time event synchronization.



ADVANTAGES & LIMITATIONS

Advantages :

- Provides instant real-time data updates.
- Reduces latency compared to traditional polling systems.
- Ensures data consistency across all connected clients.
- Improves user experience with automatic updates.
- Minimizes unnecessary network traffic.

Limitations :

- Initial setup is more complex than traditional systems.
- Requires stable internet connection.
- High number of users may require powerful servers.
- Security risks must be properly managed.
- Debugging real-time systems can be difficult.

Thank you